Mobile CRYPT-family decryption update

6 October 2026. This guide extends the earlier CRYPT14 guide included in this project.

Supported formats

Extension	Decoder and required material	Verification
.crypt	Original WhatsApp AES-192 ECB layout; built-in historical format key	SQLite structure only; no authentication
.crypt5	Legacy AES-192 CBC; supply an already-derived 24-byte AES key, or 48 hex characters	SQLite structure only; no authentication
.crypt7	AES-256 CBC; matching device key or raw AES key	SQLite structure only; no authentication
.crypt8	AES-256 CBC plus gzip; matching device key or raw AES key	Gzip and SQLite checks; no authentication
.crypt9, .crypt10, .crypt11	Historical GCM/gzip layouts; matching device key or raw AES key	Valid tag where present; old stream-only payloads remain explicitly unverified
.crypt12, .crypt14	Existing GCM layouts; matching device key or raw AES key	Authentication required; checksum verified where present
.crypt15	Existing GCM plus HKDF layouts; matching backup key	Authentication required; checksum verified where present
Other .crypt* extensions	Discovered, retained and recorded	Blocked with unsupported_crypt_version

Supported means the implemented layouts, not every historical or future variant that might use that extension. CRYPT1-4, CRYPT6, CRYPT13, CRYPT16 and other unknown variants have no decoder in this update. They are reported rather than silently skipped. An extension from a different application does not make its encryption compatible with WhatsApp.

CRYPT5 does not use the current 32-byte WhatsApp key. This version accepts its already-derived 24-byte AES key; it does not derive that key from an account email or guess account identifiers. Import the matching material through the existing key-file upload or intake hex field. The original unnumbered .crypt format uses a historical public format key; this exception is not a universal key for modern backups.

Changes to mobile extraction

1. Acquisition, extraction scope, parser routing, backup counts, browsing and case-folder testing now recognize CRYPT-family extensions through shared detection.

2. Supported database backups decode into preserved SQLite files. The existing WhatsApp parser extracts supported message, contact and related records from those databases.

3. Supported modern resource containers decode into authenticated ZIP or binary payloads. This includes non-msgstore files such as encrypted stickers or themes when their actual container and key are compatible. ZIPs are preserved without automatic unpacking; binary resource outputs are preserved as payload.bin. Resource files do not generate invented chat records or inflate msgstore backup counts.

4. Intake shows each file's outcome, matching collected key source, authentication status, export state and any reason for blocking. Legacy outputs are marked 'legacy: authenticity unverified'. Only authenticated matches count toward verified_backup_count.

5. Parsed database records also retain decryption_authenticated and decryption_validation in forensic metadata. Legacy records carry an authenticity limitation for downstream use.

6. The original encrypted files remain unchanged. The existing key acquisition workflow remains responsible for collecting private keys from accessible app data or accepting owner-supplied material.

CBC/ECB and tagless legacy stream decoding cannot establish cryptographic authenticity. Complete SQLite page layout and PRAGMA quick_check validate structure, and compressed legacy payloads must reach a clean gzip/zlib end. These checks cannot prove that a maliciously modified legacy file is authentic. Modern CRYPT12/14/15 never use an unauthenticated fallback.

Windows commands

Extract the updated project over your project working copy after backing up local configuration. Run PowerShell from the project root. Use your existing backend Python environment with its dependencies installed.

Run the focused regression suite, including all implemented formats:

```
.\Test-Crypt.ps1
```

Expected result: 122 focused tests pass. Test-Crypt14.ps1 remains compatible and now also includes the family tests.

Run the included public synthetic demo:

```
.\Test-Crypt.ps1 -CaseFolder '.\backend\tests\fixtures\crypt_family_demo' -OutputFolder '.\crypt-
family-demo-output' -ExportKeyHex
```

Expected result:

```
Backups=11 decrypted=11 blocked=0
Authenticated=7 legacy_unverified=4
```

Run your acquired data with supplied matching keys:

```
.\Test-Crypt.ps1 -CaseFolder 'D:\Evidence\Case01' -OutputFolder 'D:\Results\Case01' -KeyFile
'D:\Keys\whatsapp-key','D:\Keys\crypt5-derived-key' -ExportKeyHex
```

Remove the second key argument if the case has no CRYPT5 file. Omit -KeyFile when matching keys are already stored under recognized paths in the case folder. Omit -ExportKeyHex to avoid another secret-key file. Specify -Python 'C:\path\to\venv\Scripts\python.exe' if needed. Test-Crypt.cmd provides a command-prompt launcher.

Use a fresh output folder for each validation run. Existing output files are not removed when a later run fails, so do not treat old exported files as new successes. Output must be separate from the source folder. Windows ACLs must protect evidence and exported keys; POSIX chmod does not establish an owner-only Windows ACL.

Linux equivalent:

```
python backend/scripts/test_crypt_case.py --case-folder /evidence/case01 --output /results/case01
--key-file /keys/whatsapp-key --export-key-hex
```

For the existing Android Docker deployment, use its configured prerequisites, volumes and environment:

```
docker compose -f services/mobile-android/docker-compose.yml --project-directory . build api
frontend
docker compose -f services/mobile-android/docker-compose.yml --project-directory . up -d
docker compose -f services/mobile-android/docker-compose.yml --project-directory . exec api python
-m pytest -q tests/test_crypt_family.py tests/test_crypt14_pipeline_delivery.py
tests/test_whatsapp_crypt.py tests/test_whatsapp_crypt_v45.py tests/test_whatsapp_recovery.py
tests/test_whatsapp_key_intake.py
```

These Docker deployment commands and PowerShell launchers were not executed in this Linux validation environment. There are no new backend dependencies: ciphers use the existing cryptography provider, with optional PyCryptodome support.

Test output

File or field	Meaning
decryption_report.json	Per-file container, outcome, source/decrypted hashes, key source and validation status; no raw keys
<source-path-hash>/msgstore.db	Decoded SQLite payload
<source-path-hash>/payload.zip	Authenticated ZIP resource; not automatically extracted
<source-path-hash>/payload.bin	Authenticated binary resource
<source-path-hash>/artifacts.jsonl	Existing production parser's normalized records and forensic metadata
<source-path-hash>/matched_key.hex	Optional export of an already-collected key only after authenticated decryption
authenticated_count	Successfully decoded files that passed GCM authentication
legacy_unverified_count	Decoded legacy files without authentication

The historical folder name msgstore.db is used for any exported SQLite payload, including a contact database; the source path in the report identifies the original file. An exported CRYPT14 AES key is not a universal CRYPT15 base secret: their key derivation and key sources differ.

Exit 0 means all discovered CRYPT files decoded, including explicitly unverified legacy results. It does not mean every output authenticated. Exit 2 means at least one file was blocked or no files were found. Inspect authentication counts and per-file validation before using results. Invalid paths or unexpected operational failures can exit with an exception.

Tests run

Scenario	Result
Every implemented database format	Expected database bytes and SHA-256 returned
Legacy CBC/ECB and tagless GCM	Explicit legacy_structural_only; authenticated remains false
Wrong legacy keys	Blocked
Independent modern CRYPT12/14/15 format vectors	Passed
Modern missing/wrong key, incomplete or tampered backup	Blocked; no new decoded export
Authenticated ZIP and binary resources	Preserved; no invented chat records
Tampered modern resource	Blocked
Unknown CRYPT version with and without a key	Discovered; unsupported_crypt_version; no plaintext
Existing key capture, persistence, counts and modern chat extraction	Passed

137 relevant backend tests passed, with one existing dependency deprecation warning. These comprise test_crypt_family.py, test_crypt14_pipeline_delivery.py, test_whatsapp_crypt.py, test_whatsapp_crypt_v45.py, test_whatsapp_recovery.py, test_whatsapp_key_intake.py, test_mobile_sqlite_counts.py, test_whatsapp_modern_extract.py and test_android_mobile_coverage.py. The focused launcher runs 122 of those tests.

The CLI family demo decoded all 11 synthetic files: seven authenticated, four legacy unverified. Modern fixtures use independently generated format vectors. Legacy fixtures are synthetic examples constructed from documented layouts using a separate encryption provider; they are not backups acquired from real legacy phones.

The Android frontend TypeScript/Vite build passed. Existing bundle-size and dynamic-import warnings remain. Python compilation passed.

A broader preview/inventory/scope run had 55 passes and five failures. The exact same failures reproduced on the untouched baseline: one Chrome-cache scope expectation, two missing python-docx dependencies, and two tests needing the missing zstandard dependency. The separate handbook scope suite could not collect because zstandard was absent. The earlier guide documents two existing queue-routing assertion failures. These failures were not fixed as part of this decryption change.

Remaining limits

A CRYPT file does not generally contain the key needed to decrypt it. No new key recovery or brute force is included. Each backup may need its own key. Protected Android app keys require the existing authorized acquisition access; shared-storage-only extraction usually omits private app keys.

Unknown formats need a sample, its actual format specification and appropriate key material before a decoder can be implemented and validated. This update cannot promise decryption of all future CRYPT versions, other apps' encryption, device encryption, every deleted record, or every attachment. It extracts supported records present in decoded databases and preserves resource payloads.

Both the parser and payload decompression use a 768 MB ceiling per file/payload. The offline utility reads whole input files into memory before decoding. Large backups need adequate memory and may require a streaming implementation. Live-phone acquisition, a deployed database/object store and real customer backups were not validated in this task.

Protocol references

- Modern containers: https://github.com/ElDavoo/wa-crypt-tools
- Legacy layouts: https://github.com/den4uk/andriller/blob/master/andriller/decrypts.py
- CRYPT7 key and IV layout: https://github.com/andreas-mausch/whatsapp-viewer/blob/master/source/WhatsApp/Crypt7.cpp

The new decoders implement the described layouts independently. No upstream decoder source was copied into the project.